

DEFCON-D DDoS Platform

Professor Presentation Guide & Comprehensive Reference Manual

1. Project Overview & Tech Stack

DEFCON-D is an interactive, sandbox-style cybersecurity range built for teaching, demonstrating, and mitigating **Distributed Denial of Service (DDoS)** attacks in real time. Rather than using complex pre-built security packages, this system implements low-level network server routines, math algorithms, and responsive firewalls directly in clean Python modules.

■ Explain Like I'm 10: What is a DDoS Attack?

Imagine you run a popular ice cream shop. A bully wants to ruin your business, so they hire 100 people to fill your shop and stand in line forever without buying anything. When real, hungry customers try to get in, the shop is so packed that they can't even open the door! That is a DDoS attack. The bully is a hacker, the fake shoppers are computers, and your ice cream shop is a server.

The Technical Stack

Our system uses the following technologies to model a complete production lifecycle:

Component	Technology	Why We Used It
Core Language	Python 3.12+	Excellent standard libraries for multithreading, sockets, and math modules.
TCP Server	Python socket module	Enables low-level socket bind/listen/accept, measuring exact request latency
Web / APIs	Flask	Creates lightweight REST routes for simulator actions and serving HTML templates
Real-time Pipeline	Flask-SocketIO / WebSockets	Streams server alerts, traffic statistics, and system logs to UI every 1.0s.
Database Store	SQLite 3	Stores metrics, threat history, and firewall logs in local relational tables.
PDF Generation	ReportLab	Compiles live metrics and generates custom research reports directly on the server

2. How the Backend Systems Work

The project is structured into modular layers, separating traffic logic, logging, anomaly detection, and mitigation scripts. This ensures clean maintainability and matches modern system design principles.

■ Explain Like I'm 10: What does each part do?

- **The TCP Server:** *The gatekeeper. It accepts customers and asks them what they want.*
- **The Simulator:** *The actor. It creates the 'bully' computers that pretend to be normal customers.*
- **The Database:** *The memory. It writes down everything that happens in a digital notebook.*
- **The Detection Engine:** *The detective. It watches how fast customers arrive and reports if they are suspicious.*
- **The Mitigation Engine:** *The bouncer. It blocks bad customers from entering and locks the door for spammy IPs.*

Backend Component Breakdown

- **Target Server (server/target_server.py):** Creates a raw socket server bound to port 8080. When a connection is accepted, it starts a client thread to process the request, returning HTTP responses. It also has a background daemon that aggregates performance logs.
- **Connection Manager (server/connection_manager.py):** A thread-safe metrics accumulator. It locks statistics buffers during updates to prevent concurrency race conditions.
- **Traffic Generator (simulator/traffic_generator.py):** Spawns virtual clients using a ThreadPoolExecutor. It uses randomized delay jitters to prevent synthetic syncing, and includes safety guardrails to block target URLs targeting public domains.
- **Database Schema (database/database.py):** Automatically initializes SQLite tables. Saves metrics, security alerts, and mitigation logs with indexed relational keys.

Target Server Sockets Setup:

```
self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
self.server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
self.server_socket.bind((self.host, self.port))
self.server_socket.listen(128)
# Spawns background listener thread
self.listener_thread = threading.Thread(target=self._listen_loop, daemon=True)
```

3. Anomaly Detection and Mathematics

Rather than matching known signatures, DEFCON-D analyzes traffic dynamics every second. It evaluates three core indicators: overall Request Rates, Sudden Burst Spikes, and Shannon Entropy.

■ Explain Like I'm 10: Shannon Entropy & IP Diversity

Imagine you have a bag of 10 candies.

Case A (High Surprise / Diverse legitimate traffic): *You have 10 candies, each a different color. If you pull one out, you have no idea what color it will be! It is a big surprise. This is high entropy. High entropy is healthy—it means lots of different people are using your website.*

Case B (Zero Surprise / Targeted attack): *You have 10 candies, but ALL 10 are red. If you pull one out, it is guaranteed to be red! There is zero surprise. This is low entropy. Low entropy means a single IP is spamming the server, triggering a Red alert.*

Mathematical Formulations

1. Moving Average Burst Detection: Compares current requests/sec (PPS) against a rolling 10-second history window. If traffic exceeds this moving average by a factor of **2.5x**, an anomaly alert is triggered.

2. Shannon Entropy of Source IP Distribution: Measures randomness bits. Lower values indicate extremely concentrated source IP ranges (focused flood attacks):

Shannon Entropy Formula:

$$H(X) = - \sum P(x_i) * \log_2(P(x_i))$$

Where $P(x_i)$ is the probability (ratio) of requests from a specific source IP inside the current second.

4. Defensive Mitigation Strategies

When an anomaly or signature is detected, DEFCON-D can deploy three firewall countermeasures automatically to defend the target system resources:

■ Explain Like I'm 10: Mitigation Systems

- **Connection Limit:** *A family rule. A family can only order 8 ice creams at a time. If they try to buy 9, they are kicked out.*
- **Rate Limiter:** *A pacing rule. You can only ask for ice cream once every 10 seconds. If you ask faster, we say 'Wait!'.*
- **Firewall Blacklist:** *The bouncer's notebook. If you break the pacing or family rules, the bouncer writes your name down. For the next 30 seconds, if you walk up to the door, the bouncer locks it before you can say a word.*

Mitigation Algorithms

- **Sliding-Window Rate Limiting (mitigation/rate_limiter.py):** Monitors request frequency per IP. Uses standard lock synchronization to manage timestamps within the last 1.0s window. Rejects requests exceeding 15 req/s with HTTP 429.
- **Concurrency Limiting (mitigation/connection_limiter.py):** Restricts open TCP sockets per client IP to a maximum of 8 active sessions, protecting system socket descriptors from exhaustion.
- **Blacklist Manager (mitigation/blacklist_manager.py):** A temporary IP blocker. Connections from blacklisted IPs are dropped instantly at the listener socket accept stage. Features auto-aging unblock threads to clean up rules automatically once timers expire.

Sliding-Window Implementation Snippet:

```
now = time.time()
with self.lock:
    # Purge timestamps older than 1.0 second
    self.request_windows[ip] = [t for t in self.request_windows[ip] if now - t < 1.0]
    if len(self.request_windows[ip]) >= self.limit:
        return True # Rate limited!
    self.request_windows[ip].append(now)
    return False
```

5. Presentation Flow & Question Prep

Here are key questions your professors are likely to ask during the project defense, along with the exact structured answers you should provide.

Q: Why did you write a custom TCP socket server instead of using frameworks like Django or FastAPI?

A: Frameworks abstract away the low-level transport layer. By writing a raw socket handler in Python (`socket.socket``), we can accept connections, measure processing latencies, and drop malicious packets immediately at the socket layer before they consume application-level memory pools.

Q: How does Shannon Entropy benefit DDoS detection compared to standard threshold metrics?

A: Standard threshold filters (like PPS limiters) cannot differentiate between a legitimate traffic spike (such as a popular news release) and an attack flood. Shannon Entropy calculates IP distribution diversity. A high volume spread across thousands of distinct users maintains high entropy, whereas an attack flood from a concentrated group of malicious sources drops entropy near zero, enabling highly accurate anomaly detection.

Q: How does the system handle high-concurrency request checks without suffering from race conditions?

A: Our telemetry trackers, sliding windows, and blacklist lists utilize thread-safe locks (`threading.Lock()``). By enforcing mutually exclusive access to shared states, we prevent concurrent socket handler threads from corrupting counters or misapplying rules.

Q: Is this platform production-ready? What would you change for enterprise scale?

A: This is an educational range. To deploy this at scale in production, we would:

1. Offload IP blocking to kernel-level filters using `eBPF` (Extended Berkeley Packet Filter) or `XDP` (eXpress Data Path).
2. Replace in-memory dictionaries with `Redis` to synchronize blocklists across multiple server replicas.
3. Transition the thread-per-connection pattern to an asynchronous event loop (e.g. `asyncio``) to handle millions of connections.